

6주 차 보고서

(2026년 2 월 01 일 ~ 2026년 2 월 07 일)

작성자

배주환

이번 주 회의 내용

이번 주 한 일

```
class MemoryPool;

/*-----
   Memory
-----*/

class Memory
{
    enum
    {
        // ~1024까지 32단위, ~2048까지 128단위, ~4096까지 256단위
        POOL_COUNT = (1024 / 32) + (1024 / 128) + (2048 / 256),
        MAX_ALLOC_SIZE = 4096
    };

public:
    Memory();
    ~Memory();

    void* Allocate(int32 size);
    void Release(void* ptr);

private:
    vector<MemoryPool*> _pools;

    // 메모리 크기 <-> 메모리 풀
    // 0(1) 빠르게 찾기 위한 테이블
    MemoryPool* _poolTable[MAX_ALLOC_SIZE + 1];
};
```

```
Memory::Memory()
{
    int32 size = 0;
    int32 tableIndex = 0;

    for (size = 32; size <= 1024; size += 32)
    {
        MemoryPool* pool = new MemoryPool(size);
        _pools.push_back(pool);

        while (tableIndex <= size)
        {
            _poolTable[tableIndex] = pool;
            tableIndex++;
        }
    }

    for (; size <= 2048; size += 128)
    {
        MemoryPool* pool = new MemoryPool(size);
        _pools.push_back(pool);

        while (tableIndex <= size)
        {
            _poolTable[tableIndex] = pool;
            tableIndex++;
        }
    }

    for (; size <= 4096; size += 256)
    {
        MemoryPool* pool = new MemoryPool(size);
        _pools.push_back(pool);

        while (tableIndex <= size)
        {
            _poolTable[tableIndex] = pool;
            tableIndex++;
        }
    }
}
```

```

Memory::~Memory()
{
    for (MemoryPool* pool : _pools)
        delete pool;

    _pools.clear();
}

void* Memory::Allocate(int32 size)
{
    MemoryHeader* header = nullptr;
    const int32 allocSize = size + sizeof(MemoryHeader);

    if (allocSize > MAX_ALLOC_SIZE)
    {
        // 메모리 풀링 최대 크기를 벗어나면 일반 할당
        header = reinterpret_cast<MemoryHeader*>(::malloc(allocSize));
    }
    else
    {
        // 메모리풀에서 꺼내온다
        header = _poolTable[allocSize]->Pop();
    }

    return MemoryHeader::AttachHeader(header, allocSize);
}

void Memory::Release(void* ptr)
{
    MemoryHeader* header = MemoryHeader::DetachHeader(ptr);

    const int32 allocSize = header->allocSize;
    ASSERT_CRASH(allocSize > 0);

    if (allocSize > MAX_ALLOC_SIZE)
    {
        // 메모리 풀링 최대 크기를 벗어나면 일반 해제
        ::free(header);
    }
    else
    {
        // 메모리 풀에 반납한다
        _poolTable[allocSize]->Push(header);
    }
}

```

```

struct MemoryHeader
{
    // [MemoryHeader][Data]
    MemoryHeader(int32 size) : allocSize(size) { }

    static void* AttachHeader(MemoryHeader* header, int32 size)
    {
        new(header)MemoryHeader(size); // placement new
        return reinterpret_cast<void*>(&header);
    }

    static MemoryHeader* DetachHeader(void* ptr)
    {
        MemoryHeader* header = reinterpret_cast<MemoryHeader*>(ptr) - 1;
        return header;
    }

    int32 allocSize;
    // TODO : 필요한 추가 정보
};

/*-----
|      MemoryPool
|-----*/

class MemoryPool
{
public:
    MemoryPool(int32 allocSize);
    ~MemoryPool();

    void Push(MemoryHeader* ptr);
    MemoryHeader* Pop();

private:
    int32 _allocSize = 0;
    atomic<int32> _allocCount = 0;

    USE_LOCK;
    queue<MemoryHeader*> _queue;
};

```

```

/*-----
 *      MemoryPool
 *-----*/

MemoryPool::MemoryPool(int32 allocSize) : _allocSize(allocSize)
{
}

MemoryPool::~MemoryPool()
{
    while (_queue.empty() == false)
    {
        MemoryHeader* header = _queue.front();
        _queue.pop();
        ::free(header);
    }
}

void MemoryPool::Push(MemoryHeader* ptr)
{
    WRITE_LOCK;
    ptr->allocSize = 0;

    // Pool에 메모리 반납
    _queue.push(ptr);

    _allocCount.fetch_sub(1);
}

```

```

MemoryHeader* MemoryPool::Pop()
{
    MemoryHeader* header = nullptr;

    {
        WRITE_LOCK;
        // Pool에 여분이 있는지?
        if (_queue.empty() == false)
        {
            // 있으면 하나 꺼내온다
            header = _queue.front();
            _queue.pop();
        }
    }

    // 없으면 새로 만든다
    if (header == nullptr)
    {
        header = reinterpret_cast<MemoryHeader*> (::malloc(_allocSize));
    }
    else
    {
        ASSERT_CRASH(header->allocSize == 0);
    }

    _allocCount.fetch_add(1);

    return header;
}

```

이번주에는 성능 최적화를 위한 메모리 풀 방식에 대해서 공부했다.

Memory클래스는 시스템 전체의 메모리 할당을 관리하는 최상위 인터페이스 역할을 한다. Allocate는 요청된 크기

에 따라 적절한 할당 방식을 선택하고, MAX_ALLOC_SIZE 이하의 작은 객체는 MemoryPool을 통해 할당 받아 파편화와 속도를 잡는다. 그보다 큰 객체는 malloc을 사용한다. _pools라는 배열을 가지고 있어 요 된 사이즈에 딱 맞는 메모리 풀을 인덱스로 찾아 연결해준다.

MemoryPool클래스는 실제 메모리 파편화를 방지하고 할당 속도를 높이는 핵심 엔진이다. 메모리가 필요할 때마다 시스템에 요청하는 게 아니라, 큰 덩어리를 미리 확보해두고 확보한 덩어리를 일정한 크기의 슬롯으로 나눈다. 그리고 사용 중이지 않은 메모리 블록들은 연결 리스트 구조로 관리한다.

```
// -----  
//      1차 시도  
// -----  
  
//struct SListEntry  
//{  
//  SListEntry* next;  
//};  
//  
//struct SListHeader  
//{  
//  SListEntry* next = nullptr;  
//};  
//  
//void InitializeHead(SListHeader* header);  
//void PushEntrySList(SListHeader* header, SListEntry* entry);  
//SListEntry* PopEntrySList(SListHeader* header);
```



```
// -----
//      1차 시도
// -----
/*
void InitializeHead(SListHeader* header)
{
    header->next = nullptr;
}

void PushEntrySList(SListHeader* header, SListEntry* entry)
{
    entry->next = header->next;
    header->next = entry;
}

SListEntry* PopEntrySList(SListHeader* header)
{
    SListEntry* first = header->next;

    if (first != nullptr)
        header->next = first->next;

    return first;
}
*/
```

```
// -----
//      2차 시도
// -----

//struct SListEntry
//{
//    SListEntry* next;
//};
//
//struct SListHeader
//{
//    SListEntry* next = nullptr;
//};
//
//void InitializeHead(SListHeader* header);
//void PushEntrySList(SListHeader* header, SListEntry* entry);
//SListEntry* PopEntrySList(SListHeader* header);
```

```

// -----
//      2차 시도
// -----
/*
void InitializeHead(SListHeader* header)
{
    header->next = nullptr;
}

void PushEntrySList(SListHeader* header, SListEntry* entry)
{
    entry->next = header->next;
    while (::InterlockedCompareExchange64((int64*)&header->next, (int64)entry, (int64)entry->next) == 0)
    {
    }
}

SListEntry* PopEntrySList(SListHeader* header)
{
    SListEntry* expected = header->next;

    // ABA Problem

    while (expected && ::InterlockedCompareExchange64((int64*)&header->next, (int64)expected->next, (int64)expected) == 0)
    {
    }

    return expected;
}
*/

```



```

// -----
//      3차 시도
// -----

DECLSPEC_ALIGN(16)
struct SListEntry
{
    SListEntry* next;
};

DECLSPEC_ALIGN(16)
struct SListHeader
{
    SListHeader()
    {
        alignment = 0;
        region = 0;
    }

    union
    {
        struct
        {
            uint64 alignment;
            uint64 region;
        } DUMMYSTRUCTNAME;

        struct {
            uint64 depth : 16;
            uint64 sequence : 48;
            uint64 reserved : 4;
            uint64 next : 60;
        } HeaderX64;
    };
};

void InitializeHead(SListHeader* header);
void PushEntrySList(SListHeader* header, SListEntry* entry);
SListEntry* PopEntrySList(SListHeader* header);

```

```

// 3차 시도
// -----

void InitializeHead(SListHeader* header)
{
    header->alignment = 0;
    header->region = 0;
}

void PushEntrySList(SListHeader* header, SListEntry* entry)
{
    SListHeader expected = {};
    SListHeader desired = {};

    // 16바이트 정렬
    desired.HeaderX64.next = (((uint64)entry) >> 4);

    while (true)
    {
        expected = *header;

        // 이 사이에 변경될 수 있다
        entry->next = (SListEntry*)(((uint64)expected.HeaderX64.next) << 4);
        desired.HeaderX64.depth = expected.HeaderX64.depth + 1;
        desired.HeaderX64.sequence = expected.HeaderX64.sequence + 1;

        if (::InterlockedCompareExchange128((int64*)header, desired.region, desired.alignment, (int64*)&expected) == 1)
            break;
    }
}

SListEntry* PopEntrySList(SListHeader* header)
{
    SListHeader expected = {};
    SListHeader desired = {};
    SListEntry* entry = nullptr;

    while (true)
    {
        expected = *header;

        entry = (SListEntry*)(((uint64)expected.HeaderX64.next) << 4);
        if (entry == nullptr)
            break;

        desired.HeaderX64.next = ((uint64)entry->next) >> 4;
        desired.HeaderX64.depth = expected.HeaderX64.depth - 1;
        desired.HeaderX64.sequence = expected.HeaderX64.sequence + 1;

        if (::InterlockedCompareExchange128((int64*)header, desired.region, desired.alignment, (int64*)&expected) == 1)
            break;
    }

    return entry;
}

```

1차 시도에서는 메모리가 필요할때마다 malloc이나 new를 호출하고, 다 쓰면 바로 시스템에 반납한다.

2차 시도에서는 미리 크게 잡아놓고 쪼개 쓰기를 한다. Queue나 stack과 같은 자료구조를 사용하여 해제된 메모리를 버리지 않고 보관한다.

3차 시도에서는 일반적인 lock대신 InterlockedPushEntrySList같은 함수를 사용한다.

```

template<typename Type> <T> ✖ Type=Knight ✎
class ObjectPool
{
public:
    template<typename... Args>
    static Type* Pop(Args&&... args)
    {
#ifdef _STOMP
        MemoryHeader* ptr = reinterpret_cast<MemoryHeader*>(StompAllocator::Alloc(s_allocSize));
        Type* memory = static_cast<Type*>(MemoryHeader::AttachHeader(ptr, s_allocSize));
#else
        Type* memory = static_cast<Type*>(MemoryHeader::AttachHeader(s_pool.Pop(), s_allocSize));
#endif
        new(memory)Type( forward<Args>(args)...);    // placement new
        return memory;
    }

    static void Push(Type* obj)
    {
        obj->~Type();
#ifdef _STOMP
        StompAllocator::Release(MemoryHeader::DetachHeader(obj));
#else
        s_pool.Push(MemoryHeader::DetachHeader(obj));
#endif
    }

    static shared_ptr<Type> MakeShared()
    {
        shared_ptr<Type> ptr = { Pop(), Push };
        return ptr;
    }

private:
    static int32 s_allocSize;
    static MemoryPool s_pool;
};

template<typename Type>
int32 ObjectPool<Type>::s_allocSize = sizeof(Type) + sizeof(MemoryHeader);

template<typename Type>
MemoryPool ObjectPool<Type>::s_pool{ s_allocSize };

```

```

class Knight
{
public:
    int32 _hp = rand() % 1000;
};

class Monster
{
public:
    int64 _id = 0;
};

int main()
{
    Knight* knights[100];

    for (int32 i = 0; i < 100; i++)
        knights[i] = ObjectPool<Knight>::Pop();

    for (int32 i = 0; i < 100; i++)
    {
        ObjectPool<Knight>::Push(knights[i]);
        knights[i] = nullptr;
    }

    shared_ptr<Knight> sptr = ObjectPool<Knight>::MakeShared();
    shared_ptr<Knight> sptr2 = MakeShared<Knight>();

    for (int32 i = 0; i < 5; i++)
    {
        GThreadManager->Launch([]()
        {
            while (true)
            {
                Knight* knight = xnew<Knight>();

                cout << knight->_hp << endl;

                this_thread::sleep_for(10ms);

                xdelete(knight);
            }
        });
    }

    GThreadManager->Join();
}

```

빈번한 메모리 할당/해제 비용을 줄이기 위해 미리 메모리를 확보해두고 재사용하는 objectPool이다.

_STOMP가 켜져 있으면 디버깅을 위해 전에 만들어두었던 StompAllocator를 쓰고 아니면 기존 풀에서 메모리를 가져온다. new(memory)Type(forward<Args>(args)...)를 이용하여 이미 할당된 메모리 주소에 객체의 생성자만 호출하여 객체를 생성한다.

MakeShared()함수는 shared_ptr을 생성할 때 두번째 인자로 커스텀 삭제자를 전달한다. 그래서 shared_ptr의 참조횟수가 0이 되어 객체를 지워야 할 때, delete를 하지 않고 자동으로 ObjectPool::Push를 호출해 풀로 반환하게 만든다.

```

#pragma region TypeList
template<typename... T>
struct TypeList;

template<typename T, typename U>
struct TypeList<T, U>
{
    using Head = T;
    using Tail = U;
};

template<typename T, typename... U>
struct TypeList<T, U...>
{
    using Head = T;
    using Tail = TypeList<U...>;
};
#pragma endregion

```

```

#pragma region Length
template<typename T>
struct Length;

template<>
struct Length<TypeList<>>
{
    enum { value = 0 };
};

template<typename T, typename... U>
struct Length<TypeList<T, U...>>
{
    enum { value = 1 + Length<TypeList<U...>>::value };
};
#pragma endregion

```

```

#pragma region TypeAt
template<typename TL, int32 index>
struct TypeAt;

template<typename Head, typename... Tail>
struct TypeAt<TypeList<Head, Tail...>, 0>
{
    using Result = Head;
};

template<typename Head, typename... Tail, int32 index>
struct TypeAt<TypeList<Head, Tail...>, index>
{
    using Result = typename TypeAt<TypeList<Tail...>, index - 1>::Result;
};

#pragma endregion

```

```

#pragma region IndexOf
template<typename TL, typename T>
struct IndexOf;

template<typename... Tail, typename T>
struct IndexOf<TypeList<T, Tail...>, T>
{
    enum { value = 0 };
};

template<typename T>
struct IndexOf<TypeList<>, T>
{
    enum { value = -1 };
};

template<typename Head, typename... Tail, typename T>
struct IndexOf<TypeList<Head, Tail...>, T>
{
private:
    enum { temp = IndexOf<TypeList<Tail...>, T>::value };
public:
    enum { value = (temp == -1) ? -1 : temp + 1 };
};

#pragma endregion

```



```

#pragma region Conversion
template<typename From, typename To>
class Conversion
{
private:
    using Small = __int8;
    using Big = __int32;

    static Small Test(const To&) { return 0; }
    static Big Test(...) { return 0; }
    static From MakeFrom() { return 0; }

public:
    enum
    {
        exists = sizeof(Test(MakeFrom())) == sizeof(Small)
    };
};
#pragma endregion

```

```

#pragma region TypeCast
template<int32 v>
struct Int2Type
{
    enum { value = v };
};

template<typename TL>
class TypeConversion
{
public:
    enum
    {
        length = Length<TL>::value
    };

    TypeConversion()
    {
        MakeTable<Int2Type<0>(), Int2Type<0>()>();
    }

    template<int32 i, int32 j>
    static void MakeTable<Int2Type<i>, Int2Type<j>()>()
    {
        using FromType = typename TypeAt<TL, i>::Result;
        using ToType = typename TypeAt<TL, j>::Result;

        if (Conversion<const FromType*, const ToType*>::exists)
            s_convert[i][j] = true;
        else
            s_convert[i][j] = false;

        MakeTable<Int2Type<i>(), Int2Type<j + 1>()>();
    }

    template<int32 i>
    static void MakeTable<Int2Type<i>, Int2Type<length>()>()
    {
        MakeTable<Int2Type<i + 1>(), Int2Type<0>()>();
    }
};

```



```

template<int j>
static void MakeTable( Int2Type<length>, Int2Type<j>)
{
}

static inline bool CanConvert(int32 from, int32 to)
{
    static TypeConversion conversion;
    return s_convert[from][to];
}

public:
    static bool s_convert[length][length];
};

template<typename TL>
bool TypeConversion<TL>::s_convert[ length][ length];

template<typename To, typename From>
To TypeCast(From* ptr)
{
    if (ptr == nullptr)
        return nullptr;

    using TL = typename From::TL;

    if (TypeConversion<TL>::CanConvert(ptr->_typeid, IndexOf<TL, remove_pointer_t<To>>::value))
        return static_cast<To>(ptr);

    return nullptr;
}

template<typename To, typename From>
shared_ptr<To> TypeCast(shared_ptr<From> ptr)
{
    if (ptr == nullptr)
        return nullptr;

    using TL = typename From::TL;

    if (TypeConversion<TL>::CanConvert(ptr->_typeid, IndexOf<TL, remove_pointer_t<To>>::value))
        return static_pointer_cast<To>(ptr);

    return nullptr;
}

```

```

template<typename To, typename From>
bool CanCast(From* ptr)
{
    if (ptr == nullptr)
        return false;

    using TL = typename From::TL;
    return TypeConversion<TL>::CanConvert(ptr->_typeid, IndexOf<TL, remove_pointer_t<To>>::value);
}

```

```

template<typename To, typename From>
bool CanCast(shared_ptr<From> ptr)
{
    if (ptr == nullptr)
        return false;

    using TL = typename From::TL;
    return TypeConversion<TL>::CanConvert(ptr->_typeid, IndexOf<TL, remove_pointer_t<To>>::value);
}

#pragma endregion

```

```

#define DECLARE_TL using TL = TL; int32 _typeid;
#define INIT_TL(Type) _typeid = IndexOf<TL, Type>::value;

```

```

using TL = TypeList<class Player, class Mage, class Knight, class Archer>;

```

```

class Player
{
public:
    Player()
    {
        INIT_TL(Player);
    }
    virtual ~Player() {}

    DECLARE_TL
};

class Knight : public Player
{
public:
    Knight() { INIT_TL(Knight); }
};

class Mage : public Player
{
public:
    Mage() { INIT_TL(Mage); }
};

class Archer : public Player
{
public:
    Archer() { INIT_TL(Archer); }
};

class Dog
{
};

```

```

int main()
{
    //TypeList<Mage, Knight>::Head whoAmI;
    //TypeList<Mage, Knight>::Tail whoAmI2;

    //TypeList<Mage, TypeList<Knight, Archer>>::Head whoAmI3;
    //TypeList<Mage, TypeList<Knight, Archer>>::Tail::Head whoAmI4;
    //TypeList<Mage, TypeList<Knight, Archer>>::Tail::Head whoAmI5;

    //int32 len1 = Length<TypeList<Mage, Knight>>::value;
    //int32 len2 = Length<TypeList<Mage, Knight, Archer>>::value;

    //TypeAt<TL, 0>::Result whoAmI6;
    //TypeAt<TL, 1>::Result whoAmI7;
    //TypeAt<TL, 2>::Result whoAmI8;

    //int32 index1 = IndexOf<TL, Mage>::value;
    //int32 index2 = IndexOf<TL, Archer>::value;
    //int32 index3 = IndexOf<TL, Dog>::value;

    //bool canConvert1 = Conversion<Player, Knight>::exists;
    //bool canConvert2 = Conversion<Knight, Player>::exists;
    //bool canConvert3 = Conversion<Knight, Dog>::exists;

    //{
    //    Player* player = new Knight();

    //    bool canCast = CanCast<Knight*>(player);
    //    Knight* knight = TypeCast<Knight*>(player);

    //    delete player;
    //}

    {
        shared_ptr<Knight> knight = MakeShared<Knight>();

        shared_ptr<Player> player = TypeCast<Player>(knight);
        bool canCast = CanCast<Player>(knight);
    }

    for (int32 i = 0; i < 5; i++)
    {
        GThreadManager->Launch([]()
        {
            while (true)
            {
            }
        });
    }

    GThreadManager->Join();
}

```

TypeList : 타입들의 재귀적 리스트 구조

Length : TypeList의 길이 계산

TypeAt : 인덱스를 통한 타입 접근

IndexOf : 타입의 인덱스 검색

Conversion : 타입 간 변환 가능성 검사

TypeConversion : 동적 타입 변환 테이블

이렇게 역할을 하고 활용을 해보았는데 Player라는 부모 클래스와 Player를 상속받은 Knight, Mage, Archer클래스를 만들고 INIT_TL과 DECLARE_TL 매크로를 통해 TypeList 기능을 활용하였다.

TypeCast 함수는 shared_ptr과 일반 포인터 모두에 대해 안전한 타입 변환을 한다.

```

int main()
{
    WSADATA wsaData;
    if (::WSAStartup(MAKEWORD(2, 2), &wsaData) != 0)
        return 0;

    SOCKET clientSocket = ::socket(AF_INET, SOCK_STREAM, 0);
    if (clientSocket == INVALID_SOCKET)
    {
        int32 errCode = ::WSAGetLastError();
        cout << "Socket ErrorCode : " << errCode << endl;
        return 0;
    }

    SOCKADDR_IN serverAddr;
    ::memset(&serverAddr, 0, sizeof(serverAddr));
    serverAddr.sin_family = AF_INET;
    ::inet_pton(AF_INET, "127.0.0.1", &serverAddr.sin_addr);
    serverAddr.sin_port = ::htons(7777); // 80 : HTTP

    if (::connect(clientSocket, (SOCKADDR*)&serverAddr, sizeof(serverAddr)) == SOCKET_ERROR)
    {
        int32 errCode = ::WSAGetLastError();
        cout << "Connect ErrorCode : " << errCode << endl;
        return 0;
    }

    //-----
    // 연결 성공! 이제부터 데이터 송수신 가능!

    cout << "Connected To Server" << endl;

    while (true)
    {
        // TODO
        char sendBuffer[100] = "Hello World!";

        for (int32 i = 0; i < 10; i++)
        {
            int32 resultCode = ::send(clientSocket, sendBuffer, sizeof(sendBuffer), 0);
            if (resultCode == SOCKET_ERROR) { ... }
        }

        cout << "Send Data! Len = " << sizeof(sendBuffer) << endl;
    }
}

```

```

SOCKADDR_IN serverAddr;    // ipv4
::memset(&serverAddr, 0, sizeof(serverAddr));
serverAddr.sin_family = AF_INET;
serverAddr.sin_addr.s_addr = ::htonl(INADDR_ANY); // 너가 알아서 해결
serverAddr.sin_port = ::htons(7777);    // 80 : HTTP

if (::bind(listenSocket, (SOCKADDR*)&serverAddr, sizeof(serverAddr)) == SOCKET_ERROR) { ... }

// 영업 시작!
if (::listen(listenSocket, 10) == SOCKET_ERROR) { ... }

//-----

while (true)
{
    SOCKADDR_IN clientAddr;    // ipv4
    ::memset(&clientAddr, 0, sizeof(clientAddr));
    int32 addrLen = sizeof(clientAddr);

    SOCKET clientSocket = ::accept(listenSocket, (SOCKADDR*)&clientAddr, &addrLen);
    if (clientSocket == INVALID_SOCKET)
    {
        int32 errCode = ::WSAGetLastError();
        cout << "Accept ErrorCode : " << errCode << endl;
        return 0;
    }

    // 손님 입장!
    char ipAddress[16];
    ::inet_ntop(AF_INET, &clientAddr.sin_addr, ipAddress, sizeof(ipAddress));
    cout << "Client Connected! IP = " << ipAddress << endl;

    // TODO
    while (true)
    {
        char recvBuffer[1000];

        this_thread::sleep_for(1s);

        int32 recvLen = ::recv(clientSocket, recvBuffer, sizeof(recvBuffer), 0);
        if (recvLen <= 0)
        {
            int32 errCode = ::WSAGetLastError();
            cout << "Recv ErrorCode : " << errCode << endl;
            return 0;
        }

        cout << "Recv Data! Data = " << recvBuffer << endl;
        cout << "Recv Data! Len = " << recvLen << endl;
    }
}

```

한 파일안에서 두 개의 프로젝트를 한번에 여는 것은 처음이었다. DummyClinet를 만들어서 클라의 역할을 하도록 하였다. 서버와 연결을 해서 TCP 에코 서버를 만들어보았다. 확실히 네트워크게임프로그래밍을 들으니까 흐름이 정확히 이해가 된다. 클라에서 10개를 따로따로 보내도 서버에서 받을 때 한번에 10개를 받았다. 이것 통해서 TCP는 데이터의 경계를 구분할 수 있는 추가 정보를 포함해야 한다는 것을 알 수 있다.


```

void HandleError(const char* cause)
{
    int32 errCode = ::WSAGetLastError();
    cout << cause << "ErrorCode : " << errCode << endl;
}

int main()
{
    WSADATA wsaData;
    if (::WSAStartup(MAKEWORD(2, 2), &wsaData) != 0)
        return 0;

    SOCKET clientSocket = ::socket(AF_INET, SOCK_DGRAM, 0);
    if (clientSocket == INVALID_SOCKET)
    {
        HandleError("Socket");
        return 0;
    }

    SOCKADDR_IN serverAddr;
    ::memset(&serverAddr, 0, sizeof(serverAddr));
    serverAddr.sin_family = AF_INET;
    ::inet_pton(AF_INET, "127.0.0.1", &serverAddr.sin_addr);
    serverAddr.sin_port = ::htons(7777);

    //-----

    while (true)
    {
        char sendBuffer[100] = "Hello World!";

        int32 resultCode = ::sendto(clientSocket, sendBuffer, sizeof(sendBuffer), 0,
            (SOCKADDR*)&serverAddr, sizeof(serverAddr));

        if (resultCode == SOCKET_ERROR)
        {
            HandleError("SendTo");
            return 0;
        }

        cout << "Send Data! Len = " << sizeof(sendBuffer) << endl;

        SOCKADDR_IN recvAddr;
        ::memset(&recvAddr, 0, sizeof(recvAddr));
        int32 addrLen = sizeof(recvAddr);

        char recvBuffer[1000];

        int32 recvLen = ::recvfrom(clientSocket, recvBuffer, sizeof(recvBuffer), 0,
            (SOCKADDR*)&recvAddr, &addrLen);
    }
}

```



```

1 // udpServer.cpp
2
3 #include <winsock2.h>
4 #include <iostream>
5 #include <string>
6 #include <thread>
7 #include <chrono>
8
9 using namespace std;
10
11 void HandleError(const string& msg) {
12     cerr << msg << endl;
13 }
14
15 int main() {
16     WSADATA wsaData;
17     if (::WSAStartup(MAKEWORD(2, 2), &wsaData) != 0)
18         return 0;
19
20     SOCKET serverSocket = ::socket(AF_INET, SOCK_DGRAM, 0);
21     if (serverSocket == INVALID_SOCKET)
22     {
23         HandleError("Socket");
24         return 0;
25     }
26
27     SOCKADDR_IN serverAddr;
28     ::memset(&serverAddr, 0, sizeof(serverAddr));
29     serverAddr.sin_family = AF_INET;
30     serverAddr.sin_addr.s_addr = ::htonl(INADDR_ANY);
31     serverAddr.sin_port = ::htons(7777);
32
33     if (::bind(serverSocket, (SOCKADDR*)&serverAddr, sizeof(serverAddr)) == SOCKET_ERROR)
34     {
35         HandleError("Bind");
36         return 0;
37     }
38
39     while (true)
40     {
41         SOCKADDR_IN clientAddr;
42         ::memset(&clientAddr, 0, sizeof(clientAddr));
43         int32 addrLen = sizeof(clientAddr);
44
45         this_thread::sleep_for(1s);
46
47         char recvBuffer[1000];
48
49         int32 recvLen = ::recvfrom(serverSocket, recvBuffer, sizeof(recvBuffer), 0,
50                                 (SOCKADDR*)&clientAddr, &addrLen);
51
52         if (recvLen <= 0)
53         {
54             HandleError("RecvFrom");
55             return 0;
56         }
57
58         cout << "Recv Data! Data = " << recvBuffer << endl;
59         cout << "Recv Data! Len = " << recvLen << endl;
60
61         int32 errorCode = ::sendto(serverSocket, recvBuffer, recvLen, 0,
62                                   (SOCKADDR*)&clientAddr, sizeof(clientAddr));
63     }
64 }

```

UDP서버도 만들어보았다. TCP와는 다르게 connect를 안한다.

TCP와 UDP의 차이는 TCP는 신뢰성이 좋지만 속도는 느리고 분실이 일어나면 다시 전송을 한다. UDP는 신뢰성은 좋지 않지만 속도는 빠르고 분실에 대한 책임이 없다. 또 TCP는 경계의 개념이 없어서 패킷이 나뉘어져서 주고받을 수 있지만 UDP는 경계라는 개념이 있어서 패킷 단위로 주고 받는다. 그러나 TCP는 순서가 보장이 되고 UDP는 순서가 보장되지 않는다.

다음 주 할 일

블로킹 소켓과 논블로킹 소켓에 대해 자세히 알아보고, select모델, wsaeventselect모델, overlapped모델, iocp모델에 대해서 간단하게 공부할 예정이다.

특이사항